

# Wireless Optical Mouse

*PTX firmware — architecture, implementation and validation*

[nRF52833 PCA10100](#) | [PMW3389](#) | [Enhanced ShockBurst](#)

## Project Overview

This project implements a wireless optical mouse using an nRF52833 PCA10100 development kit as the transmitter (PTX) and an nRF52840 USB dongle as the receiver (PRX). The PTX interfaces with an optical motion sensor, acquires mouse movement and button data, and transmits that information to the dongle over a proprietary 2.4 GHz Enhanced ShockBurst (ESB) radio link.

The nRF52840 dongle receives the wireless mouse data, converts it into USB Human Interface Device (HID) reports, and sends those reports to the host PC. The resulting input is used by the operating system to control the on-screen pointer and can also be consumed directly by applications such as games as relative mouse-motion input.

## Hardware

### nRF52833 PCA10100 Development Kit — PTX

The transmitter is implemented on a Nordic Semiconductor nRF52833 PCA10100 development kit. The nRF52833 provides the 2.4 GHz radio used for the Enhanced ShockBurst (ESB) wireless link, an SPIM peripheral for communication with the optical sensor, hardware TIMER and RTC peripherals for active and idle scheduling, and GPIO/GPIOTE resources for mouse-button input.

In this project, the PCA10100 functions as the wireless mouse transmitter. It initializes and reads motion data from the PMW3389 optical sensor, tracks pending motion and button state, and transmits mouse data to the nRF52840 USB receiver over ESB. Active operation uses a hardware timer to provide 500  $\mu$ s scheduling opportunities, while RTC1 provides the lower-frequency timing used in idle mode.

The development kit also provides the two push buttons currently used as the left and right mouse buttons during development.

Reference: [1] Nordic Semiconductor, *nRF52833 Product Specification*; [2] Nordic Semiconductor, *nRF52833 DK / PCA10100 documentation*.

## PMW3389 Optical Motion Sensor

Mouse motion is measured using a PixArt PMW3389 optical motion sensor connected to the nRF52833 through SPI. The PTX firmware communicates with the sensor using the nRF52833 SPIM peripheral at 1 MHz in SPI Mode 3 with MSB-first transmission.

During initialization, the firmware resets the PMW3389, verifies its device-identification registers, uploads the required SROM firmware image, verifies the resulting SROM ID, and configures the sensor resolution to 800 CPI.

During normal operation, motion is acquired using the PMW3389 motion-burst interface. A single burst transaction returns the sensor status together with signed 16-bit X- and Y-axis displacement values. These relative motion deltas represent the displacement accumulated by the sensor since the preceding motion read.

The PTX adds each returned X/Y displacement to a software motion accumulator before transmission. This decouples sensor motion acquisition from individual radio transactions and allows motion that exceeds the range of one transmitted report to be preserved across subsequent reports.

Reference: [3] PixArt Imaging, *PMW3389DM-T3QU Optical Gaming Navigation Sensor Datasheet*.

## nRF52840 PCA10059 USB Receiver — PRX

The wireless receiver is implemented using a Nordic Semiconductor nRF52840 PCA10059 USB device. The nRF52840 receives mouse packets from the PTX over the same 2.4 GHz Enhanced ShockBurst (ESB) link and provides the USB device interface to the host PC.

Received ESB packets contain the current button state together with signed X- and Y-axis relative motion data. The receiver firmware accumulates incoming motion as needed and converts the resulting mouse state into USB Human Interface Device (HID) reports.

The PCA10059 connects directly to the host through USB and enumerates as a HID mouse. USB interrupt-IN reports are sent at a 1 ms interval, allowing the operating system or an application to consume the received wireless motion and button input without requiring a custom host-side driver.

Reference: [4] Nordic Semiconductor, *nRF52840 Product Specification*; [5] Nordic Semiconductor, *nRF52840 USB Dongle / PCA10059 documentation*.

## Mouse Buttons

The two onboard push buttons on the nRF52833 PCA10100 development kit are used as the left and right mouse buttons. Each button is configured as an active-low GPIO input with an internal pull-up resistor.

Button transitions are detected using the nRF52833 GPIOTE peripheral, allowing both press and release events to be handled asynchronously. The resulting button state is represented as a bit field aligned with the USB HID mouse button layout and is included in each applicable ESB mouse packet sent to the USB receiver.

Because the buttons are integrated into the development kit, no additional external button hardware is required for the current prototype.

## Architecture and Data Flow

The mouse is divided into two embedded systems connected by a proprietary 2.4 GHz ESB radio link:

**PMW3389 sensor / buttons → nRF52833 PTX → ESB radio link → nRF52840 PRX → USB HID → host PC**

The transmitter is responsible for acquiring physical mouse input and reliably transporting it over the wireless link. The receiver converts the wireless mouse state into standard USB HID reports that can be consumed by the host operating system.

### PTX Data Path

During active operation, a hardware timer generates a scheduling opportunity every 500  $\mu$ s. When a scheduled poll is pending, the nRF52833 performs a PMW3389 motion-burst read and adds the returned signed X/Y displacement to a software motion accumulator. Sensor acquisition does not wait for the ESB transmitter to become available.

Sensor reads are serviced independently of ESB busy state. Each pending scheduler event can trigger a PMW3389 motion-burst read even while a previous radio packet is still in flight, and each returned displacement is added to the software motion accumulator. This decouples the 2 kHz acquisition path from radio transaction timing; transmission remains separately gated by ESB availability.

Before transmission, the current pending motion is copied into a report-sized snapshot. The snapshot is packaged with the current left/right button state and a packet sequence number and submitted to ESB. Motion is not removed from the software accumulator when the packet is queued. Instead, the transmitted snapshot is committed only after ESB reports a successful transmission. While that packet is in flight, subsequent scheduled sensor reads continue and any newly measured displacement is added to the software accumulator for a later transmission.

Button changes are detected asynchronously through GPIOTE. A button transition marks the mouse as active and causes the updated button-state bit field to be included in the next available radio transmission.

## Wireless Transport

The PTX and PRX communicate using Nordic Semiconductor Enhanced ShockBurst at 2 Mbps. Each packet contains a sequence number, button-state field, and signed 16-bit X- and Y-axis relative motion values.

ESB automatic acknowledgment is used to provide link-level delivery feedback, while automatic retransmission is disabled (`retransmit_count = 0`). The PTX permits only one application-level mouse packet to be in flight at a time, allowing the firmware to associate the resulting ESB success or failure event with the corresponding motion and button snapshot. On a successful transmission, the transmitted motion snapshot is subtracted from the accumulator. On failure, it remains pending, so the next transmission can contain the most up-to-date accumulated motion rather than retrying the older packet.

## PRX and USB Data Path

The nRF52840 receiver accepts mouse packets from the ESB link and incorporates the received relative motion into its own pending mouse state. The wireless receive schedule and USB reporting schedule are therefore decoupled: motion arriving between USB transfers can be accumulated until the next USB report opportunity.

The receiver exposes a standard USB HID mouse interface to the host PC. At each USB reporting opportunity, pending relative X/Y motion and the current button state are formatted into a HID input report and transferred through the USB interrupt-IN endpoint.

This architecture preserves relative motion across independently scheduled sensor, radio, and USB stages while preventing a failed or delayed radio transmission from immediately discarding unreported mouse movement.

## Timing and Scheduling

The PTX uses separate timing sources for active and idle operation. During active use, `TIMER1` is configured for a 500  $\mu$ s period, giving the mouse a maximum scheduled polling and transmission rate of 2 kHz.

Each timer event marks a sensor-read opportunity as pending. The main loop services that request independently of the ESB busy state, so PMW3389 acquisition can continue at the scheduled 500  $\mu$ s interval while a radio packet is in flight. ESB transmission is handled separately and begins only when the transmitter is available. Therefore, 500  $\mu$ s is the target sensor-acquisition interval and the maximum opportunity rate for generating new mouse updates, not a guarantee that an ESB packet is transmitted every 500  $\mu$ s.

When the sensor reports motion or a button changes state, the inactivity timer is reset. After approximately 10 seconds without activity, the PTX enters idle mode. The 2 kHz hardware timer is disabled and RTC1 generates a slower polling interval of approximately 100 ms. New motion or button activity returns the scheduler to active mode and restarts the hardware timer.

The radio operates at 2 Mbps with ESB acknowledgments enabled and automatic retransmission disabled. Each application packet therefore receives a single radio attempt. The application allows only one mouse packet to be in flight at a time, but PMW3389 reads and software motion accumulation continue while that packet is in flight. When ESB becomes available, the application snapshots the current pending motion for the next transmission. Eliminating the retry delay also returns the radio to the application sooner after an unsuccessful attempt.

On the receiver side, USB HID reporting uses a 1 ms interrupt interval. The USB and ESB schedules are independent, so motion received between USB report opportunities is accumulated and included in a later HID report.

The main timing intervals are therefore:

- **Active PTX scheduling:** 500  $\mu$ s (2 kHz maximum opportunity rate)
- **Idle PTX polling:** approximately 100 ms
- **Idle timeout:** approximately 10 s
- **USB HID report interval:** 1 ms (1 kHz)
- **ESB radio bitrate:** 2 Mbps

## Firmware Modules

The firmware is divided into several small modules so that sensor communication, motion tracking, scheduling, and wireless transmission are kept separate from the main application logic.

### PTX Firmware

#### `ptx_main.c` — Application Control

`ptx_main.c` ties the PTX subsystems together. It initializes the system clocks, sensor, scheduler, ESB radio, buttons, and status LEDs, then runs the main application loop.

The main loop handles completed ESB transmissions, scheduled sensor reads, motion accumulation, button changes, idle-state transitions, and starting new radio transmissions. Interrupt handlers are used for timing, button events, and ESB events, while most application processing remains in the main context.

## **sensor.c / sensor.h — PMW3389 Driver**

The sensor module contains the low-level PMW3389 interface. It configures the nRF52833 SPIM peripheral and provides the register read/write operations needed to initialize and communicate with the sensor.

The module also handles the PMW3389 power-up/reset sequence, SROM upload and verification, CPI configuration, and motion-burst reads. Higher-level application code receives signed X/Y motion values without needing to handle the sensor's SPI transaction details directly.

## **mouse\_motion.c / mouse\_motion.h — Motion Accumulator**

The motion module maintains X- and Y-axis displacement that has been measured but not yet successfully transmitted.

New sensor motion is added to signed 32-bit accumulators. When the radio is ready, the module provides a 16-bit X/Y snapshot suitable for a mouse packet without immediately removing it from the accumulator. The snapshot is removed only after the corresponding ESB transmission succeeds.

The software accumulator retains uncommitted motion after a failed transmission. Sensor reads continue while a packet is in flight, so newly measured displacement can be added to the pending accumulator before the previous TX result is processed.

## **mouse\_scheduler.c / mouse\_scheduler.h — Active and Idle Scheduling**

The scheduler controls when the PTX should attempt to read the sensor.

During active operation, TIMER1 generates a scheduling event every 500  $\mu$ s. After the configured inactivity period, the scheduler disables the active timer and switches to slower RTC1-based polling. New motion or button activity returns the scheduler to active mode.

The scheduler exposes pending timing events to the main loop rather than performing sensor or radio operations directly from its interrupt handlers.

## **esb\_mouse\_tx.c / esb\_mouse\_tx.h — Wireless Transmission**

The ESB transmission module configures the nRF52833 radio as the PTX and owns the wireless mouse packet format.

It accepts motion and button data from the application, builds the ESB payload, and starts the radio transmission. It also tracks whether a packet is currently in flight and reports transmission success or failure back to the main application.

The ESB event handler performs the immediate radio-event processing, while the application responds to the resulting transmit status from the main loop.

## PRX Firmware

The nRF52840 receiver firmware contains two main responsibilities: receiving and decoding ESB mouse packets and presenting the resulting mouse state through USB HID.

The ESB receive path validates incoming mouse packets and extracts the packet sequence number, button state, and relative X/Y motion. Received motion is accumulated until it can be included in a USB report.

The USB HID portion defines the mouse report format and interrupt-IN endpoint used to send relative motion and button state to the host. USB transfers are scheduled independently of ESB reception, allowing multiple received motion updates to be combined when necessary.

## Wireless Packet Format

Mouse data is sent from the PTX to the PRX in a fixed 6-byte ESB application payload.

| Byte Offset | Field           | Type    | Description                                              |
|-------------|-----------------|---------|----------------------------------------------------------|
| 0           | packet_sequence | uint8_t | Sequence number used for packet tracking and diagnostics |
| 1           | buttons         | uint8_t | Bit field containing the current mouse-button state      |
| 2-3         | x               | int16_t | Signed relative X-axis motion                            |
| 4-5         | y               | int16_t | Signed relative Y-axis motion                            |

The corresponding packet structure is:

```
typedef struct __attribute__((packed))
{
    uint8_t packet_sequence;
    uint8_t buttons;
    int16_t x;
    int16_t y;
} mouse_packet_t;
```

The structure is packed so the transmitted layout is exactly 6 bytes with no compiler-added padding between fields.

The buttons field uses the same bit positions as the USB HID button field. Bit 0 represents the left mouse button and bit 1 represents the right mouse button.

The X and Y fields contain relative motion rather than absolute cursor coordinates. Each value represents motion that is still pending on the PTX and has been selected for the current transmission. The PTX removes that motion from its accumulator only after ESB reports that the packet was transmitted successfully.

The sequence number increments for each new application packet and is retained primarily as a diagnostic field. It is not required by the normal motion-delivery path, but gaps in received sequence values can be useful for identifying packets that were not acknowledged or for correlating PTX and PRX

logs.

Because automatic retransmission is disabled, each application packet is attempted once at the radio layer. The application-level sequence number remains available as a low-cost diagnostic aid even though normal mouse operation does not depend on it.

## Reliability Behavior

The PTX is designed so that mouse movement is not discarded simply because a radio transmission is delayed or unsuccessful.

When motion is ready to be sent, the current pending X/Y values are copied into a transmission snapshot. The motion is not immediately removed from the software accumulator. Instead, the PTX waits for the ESB transmission result. If the packet is transmitted successfully, the corresponding X/Y snapshot is committed and removed from the accumulator. If the transmission fails, that motion remains pending and can be included in a later packet.

Only one application-level mouse packet is allowed to be in flight at a time, but sensor acquisition is not blocked by that packet. While ESB is busy, scheduled PMW3389 reads continue and newly measured displacement is added to the software accumulator. Once the transmitter becomes available, the application takes a fresh snapshot of the accumulated pending motion for the next packet. This avoids a queue of discrete stale reports while preserving motion acquired during radio activity.

Button state is handled in a similar way. Each transmission uses a snapshot of the current button bit field. After a successful transmission, the firmware clears the pending button-change flag only if the transmitted snapshot still matches the current button state. If another button transition occurred while the packet was in flight, the change remains pending and is sent in a later packet.

The ESB link uses acknowledgments to provide link-level delivery feedback, but automatic retransmission is disabled. The application receives a transmit-success or transmit-failure event after the single radio attempt and uses that result to decide whether the associated motion can be removed from the pending state. On failure, the motion snapshot remains pending and is incorporated into a later packet rather than immediately retrying the older packet.

Together, these behaviors allow the application to tolerate short periods of radio unavailability or packet loss without immediately losing mouse movement or newer button-state changes.

## Power Behavior

The PTX uses separate active and idle operating modes to reduce unnecessary sensor polling when the mouse is not being used.

During active operation, TIMER1 generates scheduling events every 500  $\mu$ s. Motion detected by the PMW3389 or a mouse-button transition updates the scheduler's last-activity time and keeps the PTX in the active state.

If no activity is detected for approximately 10 seconds, the scheduler enters idle mode. The 500  $\mu$ s hardware timer is disabled and sensor polling is reduced to approximately once every 100 ms using RTC1. This lowers the amount of processor activity and peripheral use while still allowing the mouse to periodically check for new movement.

If motion is detected during an idle poll, or if either mouse button changes state, the scheduler immediately returns to active mode and restarts TIMER1.

The main application loop uses the processor's event-wait mechanism when no immediate work is pending rather than continuously performing application processing.

The current power-management design is intended mainly to reduce unnecessary processing during periods of inactivity. More elaborate power optimization, such as placing the optical sensor into a lower-power operating mode or using deeper nRF52 sleep states, could be added in a future battery-powered implementation.

## Testing and Results

The PTX was evaluated using manual input checks and firmware counters reported through RTT. The nRF52840 PRX and host PC were connected for functional input testing. Quantitative measurements below describe the PTX; USB report throughput was not measured.

The tested configuration used a 500  $\mu$ s active TIMER1 period, RTC1 at 32,768 Hz, an approximately 100 ms idle polling interval, and a configured 10-second inactivity timeout. ESB used 2 Mbps and 0 dBm output power with automatic retransmission disabled (`retransmit_count` = 0). The `retransmit-delay` setting remains configured but is not used when no retransmissions are permitted.

## Measured Operation Timing

Timing measurements were made with TIMER4 running at 16 MHz.

| Measurement                           | Observed value                                                 |
|---------------------------------------|----------------------------------------------------------------|
| PMW3389 motion-burst read             | ~192 $\mu$ s steady-state;<br>typically ~191.6–198 $\mu$ s     |
| First/reinitialized motion-burst read | ~417.5 $\mu$ s maximum<br>observed                             |
| ESB submit-to-TX_SUCCESS latency      | ~396.17 $\mu$ s average;<br>~392.87–397.62 $\mu$ s<br>observed |

The ESB measurement used the final 2 Mbps, 6-byte payload, acknowledgment-enabled, no-retransmission configuration. At the 2 kHz active sensor rate, the 500  $\mu$ s scheduling interval leaves approximately 308  $\mu$ s after a normal motion-burst read.

Sensor acquisition and radio transmission are asynchronous after ESB submission, so the ~192  $\mu$ s sensor-read time and ~396  $\mu$ s submit-to-success latency are not additive within the 500  $\mu$ s sensor interval. Sensor reads can continue while the radio transaction is in progress.

## Functional Input

Manual testing confirmed movement in both axes and directions, slow and fast movement, and repeated abrupt stops without visible residual cursor movement. Left-button dragging and release, right-click operation, and simultaneous movement and button input behaved as intended. These observations verify basic operation of the complete input path; they are not latency measurements.

## Active Scheduling and Sensor Throughput

A cumulative TIMER1 interrupt counter was sampled against RTC1 over approximately one-second windows. The rate was calculated as event count  $\times$  32,768 / elapsed RTC ticks. Successful sensor reads and successful ESB outcomes were counted separately.

Nine consecutive windows in the initial scheduler capture reported 2,000 Hz. Subsequent captures remained approximately 2,000 Hz, with integer output occasionally showing 1,999 or 2,001 Hz. This verifies the average serviced interrupt rate, not individual interrupt spacing or jitter.

While stationary but still active, the PTX performed approximately 2,000 successful sensor reads per second with little or no radio traffic. After sensor acquisition was decoupled from ESB busy state, movement captures also maintained approximately 1,999–2,000 successful sensor reads/s while radio

activity increased substantially. With automatic retransmission disabled, observed successful ESB rates reached approximately 1,551/s in a high-activity window, while other movement windows varied with the amount of nonzero motion.

These results distinguish sensor scheduling from radio throughput. A read returning zero motion does not necessarily cause a packet to be submitted, so radio traffic varies with movement. The full observed 2 kHz sensor-read rate is primarily a consequence of servicing scheduled PMW3389 reads independently of ESB busy state. Disabling automatic retransmission separately reduces radio occupancy after an unsuccessful attempt and allows the next current packet to be considered sooner. The observed radio rate is not a measured maximum or a USB report rate.

## Transmission Accounting and No-Retry Observations

Testing exposed a completion-handoff race. The ESB module was changed to remain busy until the main loop atomically consumes the TX result. The main loop processes that result and commits the associated snapshot before submitting another packet.

With automatic retransmission disabled, representative approximately one-second windows during active movement produced the following observations:

| Measurement                       | Observed value |
|-----------------------------------|----------------|
| Active TIMER1 rate                | 1,999–2,000 Hz |
| Sensor read rate during movement  | 1,999–2,000 Hz |
| Peak observed successful TX rate  | ~1,551 Hz      |
| Maximum radio attempts per packet | 1              |

Individual reporting windows can differ by one between submissions and completed outcomes because a submission and its completion may fall on opposite sides of the logging boundary.

During the no-retry test, high-activity windows typically showed only a small number of failed outcomes. Examples included 7 failures with 929 submissions and 11 failures with 1,434 submissions, corresponding to roughly 0.75% of submitted packets in those windows. These are observations from the specific test setup, not controlled reliability results.

The maximum-attempt counter remained at 1, confirming that each application packet received only its initial radio attempt. The successful-packet retry counter remained at zero. A failed outcome therefore indicates that no acknowledgment was obtained for that single attempt; it does not establish whether the receiver physically received the payload, nor does it imply lost cursor displacement because the associated motion remains pending for a later packet.

## PTX Motion Accounting

Signed 64-bit diagnostic totals tracked motion added to the software accumulator and motion committed after successful TX results. At each logging snapshot, both axes were checked against the full signed 32-bit pending values:

total added – total committed = pending motion

Every supplied snapshot reported `accounting_ok=1`, including reporting windows containing TX failures. Pending values during the observed movement sample were between –1 and +1 on each axis. After movement stopped, both pending values reached zero and remained zero as radio traffic ceased.

This checks consistency of the PTX accumulator accounting at the sampled points. Because the diagnostic totals follow the same add and commit operations, it does not independently verify snapshot correctness, preservation of every intermediate movement, or exactly-once application of displacement by the PRX.

## Idle and Wake Behavior

After inactivity, the log reported entry into idle mode. Complete idle windows showed zero active TIMER1 interrupts, ten successful sensor reads over 32,770 RTC ticks, and no radio submissions. This corresponds to approximately 10 reads/s; integer rate formatting displayed 9 Hz because the measured interval was slightly longer than one second.

Movement returned the scheduler to active operation, with subsequent full windows again showing approximately 2,000 timer interrupts/s and resumed transmissions. A separate left-button test also returned the PTX to active mode, and the waking click registered on the PC.

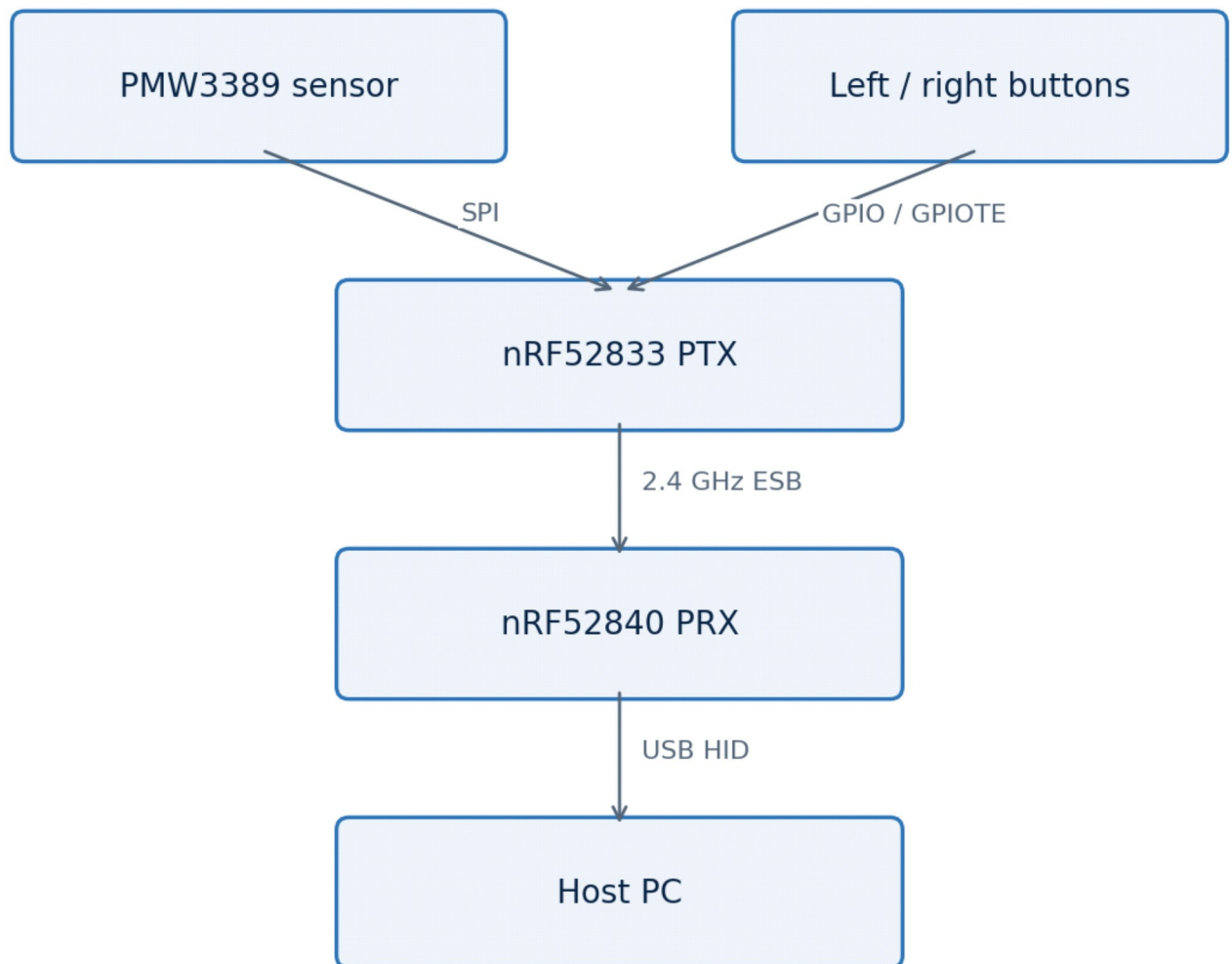
The observations confirm idle polling and motion/left-button wake behavior. The 10-second timeout was configured and idle entry was observed; its exact duration and wake latency were not independently measured. Transition windows included both active and idle time. Current consumption and right-button wake as a separate test were not measured. A system-level remote-wake test confirmed that mouse-button activity could wake a suspended host through the PRX USB interface; USB remote-wake implementation and validation are documented with the PRX firmware.

## Validation Scope

These tests provide evidence for basic input operation, average active scheduling rate, observed sensor/radio throughput, balanced TX accounting, sampled motion accounting, and PTX idle/wake behavior. They do not establish maximum throughput, end-to-end latency, radio performance across controlled environments, accumulator behavior near numeric limits, or receiver-side exactly-once motion delivery. USB timing and receiver-specific validation belong in the PRX documentation.

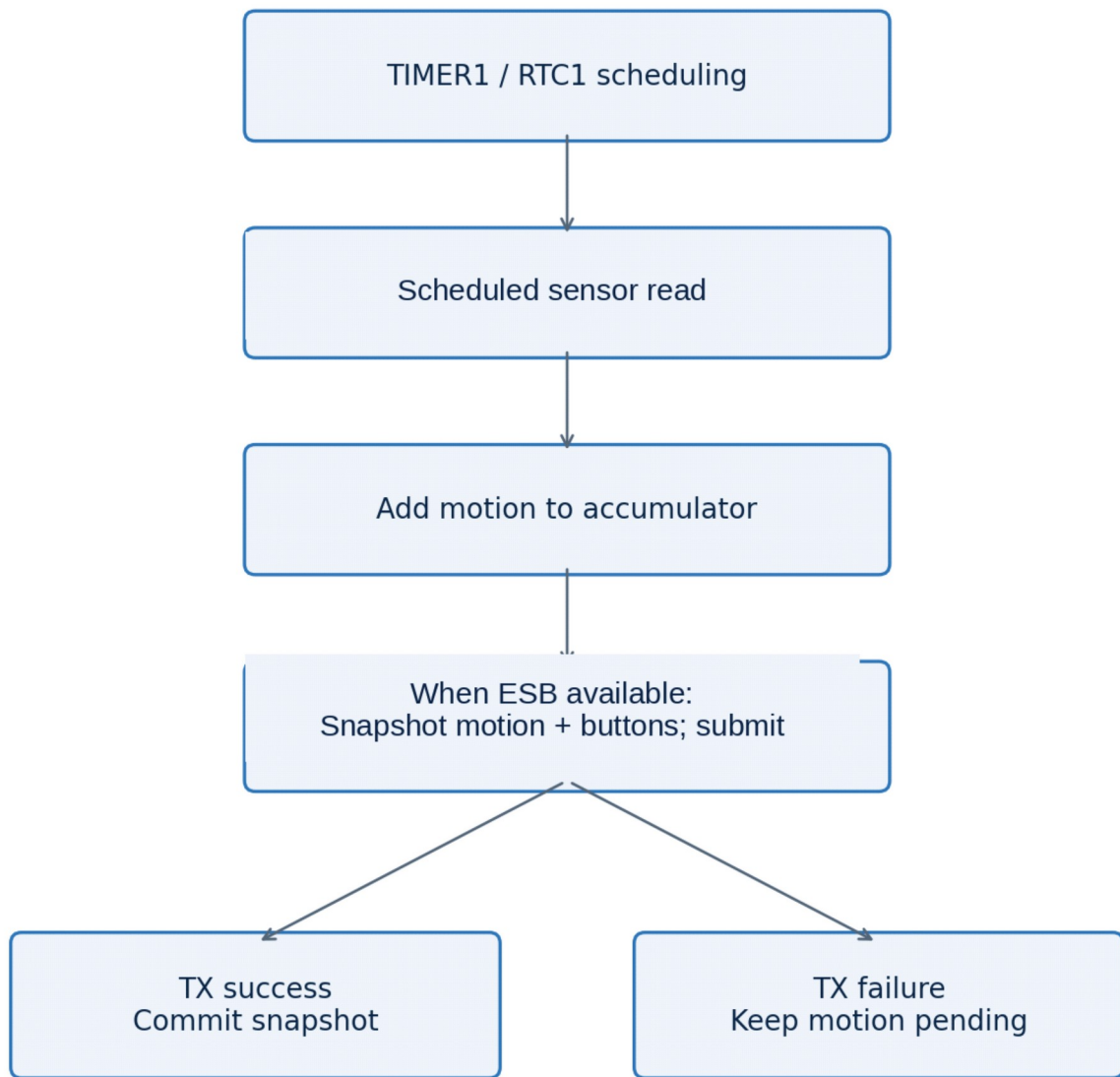
## System and Firmware Diagrams

### System overview



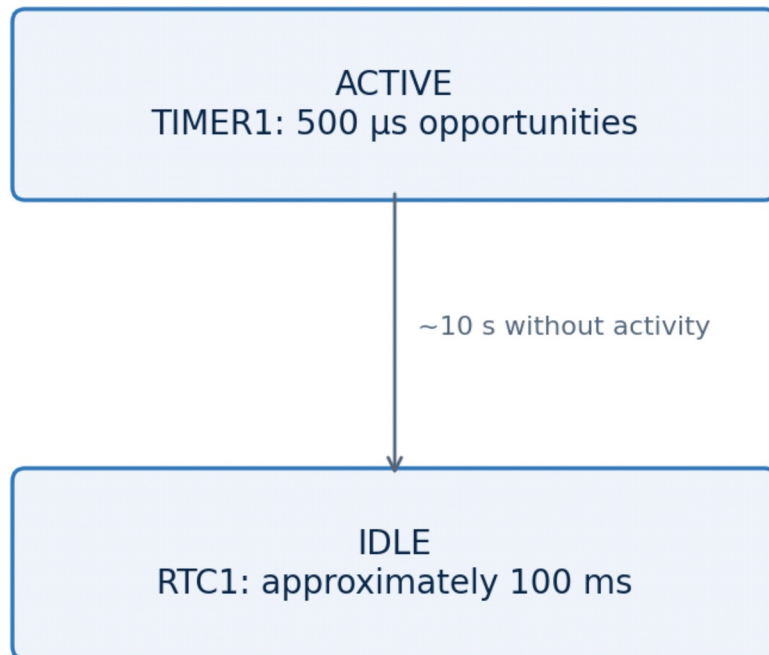
Sensor and button input reach the PC through the PTX radio link and PRX USB interface.

## PTX data flow



The main loop processes the TX outcome before accepting a new transmission.

## Scheduler states



Motion detected during an idle poll or a button event returns IDLE to ACTIVE.